

Guide pour la personne qui a fait les pages (Rôle 2)

Objectif : Savoir comment prévoir une page qui affiche les **résultats de l'API** (Rôle 4 – localisation OpenStreetMap) et comment le **state management** (Rôle 3) peut relier le tout.

1. Ce que livre la personne 4 (API / Localisation)

La personne 4 (Rôle 4 – Benjamin) travaille dans la branche `external-api` et livre :

- **Fichier :** `lib/api/location_api.dart`
- **Rôle :** Appels à l'API OpenStreetMap (Nominatim) pour la recherche d'adresses.

1.1 Ce que l'API expose (à confirmer avec Rôle 4)

En général, une API de ce type expose au moins :

Élément	Description
Fonction de recherche	Ex. <code>searchLocation(String query)</code> ou <code>searchAddress(String query)</code>
Modèle de résultat	Ex. un modèle avec <code>latitude</code> , <code>longitude</code> , <code>address</code> (ou <code>displayName</code>)

Exemple de signature (à adapter au code réel de Rôle 4) :

```
// Dans lib/api/location_api.dart (Rôle 4)
Future<LocationResult> searchLocation(String query) async {
  // Appel API OpenStreetMap Nominatim
  // Retourne lat, lng, adresse formatée
}
```

Exemple de modèle (si Rôle 4 en fournit un) :

```
// Ex. lib/models/location_model.dart (Rôle 1 ou 4)
class LocationResult {
  final double latitude;
  final double longitude;
  final String address; // ou displayName
  LocationResult({required this.latitude, required this.longitude, required
this.address});
}
```

À faire de ton côté (Rôle 2) : Demander à la personne 4 le **nom exact** du fichier, des **méthodes** et du **modèle** (ou des champs) retournés pour pouvoir les utiliser dans tes pages.

2. Quelle page prévoir pour afficher les résultats de l'API ?

Tu peux prévoir **une ou plusieurs** des options suivantes. L'idée est d'avoir un écran qui utilise les **résultats** renvoyés par l'API (liste d'adresses, lieu sélectionné, carte, etc.).

Option A – Écran « Résultats de recherche de lieu » (recommandé)

- **Fichier** : `lib/views/location_search_screen.dart` (ou `search_location_screen.dart`).
- **Rôle** :
 - Un champ de recherche (TextField) : l'utilisateur tape une adresse.
 - Au clic sur « Rechercher » (ou après debounce), on appelle l'API (via un **provider** ou un **controller**, voir plus bas).
 - La page affiche la **liste des résultats** (ex. `address`, et éventuellement lat/lng).
 - Au tap sur un résultat, on **retourne** ce lieu à l'écran précédent (ex. formulaire « Ajouter un service ») ou on navigue vers une carte.

Données à afficher par résultat (selon ce que Rôle 4 retourne) :

- Adresse (texte)
- Optionnel : latitude, longitude (pour afficher sur une carte plus tard)

Option B – Intégration dans l'écran « Ajouter / Modifier un service »

- **Fichier existant** : `lib/views/add_service_screen.dart`.
- **Rôle** :
 - Un champ « Lieu » ou « Adresse » avec un bouton « Rechercher » ou une icône.
 - Au clic, ouvre soit un **bottom sheet**, soit navigue vers l'**écran de recherche de lieu** (Option A).
 - Après sélection d'un résultat (API), afficher l'adresse choisie dans le champ et stocker lat/lng pour le `ServiceModel` (si le modèle le permet).

Ici, la **page qui affiche le résultat** de l'API peut être soit l'écran dédié (Option A), soit le bottom sheet / dialog qui liste les résultats.

Option C – Écran « Carte » (optionnel)

- **Fichier** : `lib/views/map_screen.dart`.
- **Rôle** : Afficher un marqueur sur la carte à partir des **lat/lng** retournés par l'API (Rôle 4 peut fournir lat/lng même si l'affichage carte est fait par toi).

Tu peux réutiliser les **mêmes données** (latitude, longitude, address) que sur l'écran de résultats.

3. Comment le state management peut relier (Rôle 3 – Providi)

Pour que **tes pages** affichent les résultats de l'API sans appeler Firebase ou l'API directement depuis la vue, tout passe par un **pont** : provider ou controller.

3.1 Qui fait quoi

Rôle	Fichier / responsabilité
------	--------------------------

Rôle	Fichier / responsabilité
Rôle 4	<code>lib/api/location_api.dart</code> – appelle OpenStreetMap, retourne des données (ex. liste de <code>LocationResult</code>).
Rôle 3 (State)	Expose ces données et actions à l'UI : soit un LocationProvider , soit une méthode dans un provider existant.
Toi (Rôle 2)	Écrans qui lisent les données depuis le provider et déclenchent les actions (ex. « rechercher », « sélectionner »).

3.2 Deux façons de relier

Schéma 1 – Un provider « Location » (recommandé si plusieurs écrans utilisent la recherche)

- **Rôle 3** crée `lib/providers/location_provider.dart` qui :
 - Appelle `LocationAPI` (ou le service livré par Rôle 4).
 - Garde en état : `List<LocationResult> searchResults, bool isLoading, String? error`.
 - Expose par exemple : `searchLocation(String query)`, getters pour `searchResults`, `isLoading`, `error`.
- **Rôle 3** enregistre ce provider dans `main.dart` (à côté de `AuthProvider`, etc.) et ajoute la route vers ton écran dans `app_routes.dart` (ex. `/search-location`).
- **Toi** : dans ta page (ex. `location_search_screen.dart`), tu utilises `Provider.of<LocationProvider>(context)` (ou `context.watch`) pour :
 - Lancer la recherche : `locationProvider.searchLocation(query)`.
 - Afficher `locationProvider.searchResults`, `locationProvider.isLoading`, `locationProvider.error`.

Résultat : la page affiche bien les **résultats de l'API** via le state management.

Schéma 2 – Pas de provider dédié (écran appelle un service)

- Un **controller** ou un **service** (ex. `LocationAPI` directement) est appelé depuis l'écran.
- L'écran garde en local le résultat (ex. `List<LocationResult> _results, bool _loading`) dans un `StatefulWidget` (ou avec un petit state).
- Rôle 3 n'a rien de plus à faire pour l'état « recherche », mais doit quand même **ajouter la route** vers ta nouvelle page dans `app_routes.dart` et `app.dart`.

Les deux sont valides ; le **Schéma 1** est plus cohérent avec le reste du projet (`AuthProvider`, `ServiceProvider`) et facilite la réutilisation (ex. même recherche depuis Home et depuis Add service).

4. Ce que tu dois prévoir côté « pages » (Rôle 2)

4.1 Nouvel écran à créer (recommandé)

- **Fichier** : `lib/views/location_search_screen.dart`.
- **Contenu type** :
 - `TextField` pour la requête de recherche.
 - Bouton « Rechercher » (ou recherche au debounce).

- Zone d'affichage des **résultats** :
 - Si `isLoading` → `CircularProgressIndicator` (ou ton composant de chargement).
 - Si `error != null` → message d'erreur (SnackBar ou texte).
 - Sinon → liste des résultats (ex. `ListView` de cartes ou de `ListTile` avec `address`).
- Au tap sur un résultat : soit retourner le lieu à l'écran précédent (ex. `Navigator.pop(context, selectedLocation)`), soit enregistrer dans un provider puis pop.

Tu n'as pas besoin de connaître l'implémentation de l'API ; tu as juste besoin du **type** des résultats (ex. `LocationResult` avec `address`, `latitude`, `longitude`) pour afficher les bonnes propriétés.

4.2 Intégration dans `add_service_screen.dart`

- Un bouton ou champ « Choisir le lieu » qui :
 - Ouvre `LocationSearchScreen` (via `Navigator.pushNamed(context, '/search-location')` ou `Navigator.push`).
 - À la fermeture (`Navigator.pop` avec le lieu sélectionné), tu reçois le résultat et tu mets à jour le champ « Lieu » (et si le modèle le permet, lat/lng pour le service).

Comme ça, la **page qui affiche le résultat** de l'API est soit l'écran de recherche, soit le formulaire qui affiche l'adresse sélectionnée.

5. Ce que Rôle 3 (State management) doit relier

Pour que ta page et l'API soient reliées proprement, Rôle 3 peut faire :

1. **Créer `LocationProvider`** (si vous choisissez le Schéma 1) qui :
 - Utilise `LocationAPI` (ou le nom exact du fichier / classe livrés par Rôle 4).
 - Expose au minimum : une méthode du type `searchLocation(String query)` et les données : liste de résultats, loading, erreur.
2. **Enregistrer le provider** dans `main.dart` (ex. `ChangeNotifierProvider<LocationProvider>` ou `MultiProvider`).
3. **Ajouter la route** vers ton nouvel écran (ex. `/search-location` → `LocationSearchScreen`) dans `lib/routes/app_routes.dart` et s'assurer que `app.dart` utilise ces routes (ex. `routes: AppRoutes.routes`).

Si Rôle 3 utilise un **controller** au lieu d'un provider pour la localisation, il te dira quoi utiliser (ex. `LocationController.search(query)`) et dans quel fichier ; le principe reste le même : ta page appelle une méthode et affiche des données, sans appeler l'API directement.

6. Récap pour toi (personne qui a fait les pages)

À faire	Détail
Prévoir un écran	Ex. <code>location_search_screen.dart</code> qui affiche les résultats de recherche de lieu (liste d'adresses, etc.).
Demander à Rôle 4	Le nom du fichier API, des méthodes (ex. <code>searchLocation</code>) et du modèle de résultat (champs : <code>address</code> , <code>lat</code> , <code>lng</code> , etc.).

À faire	Détail
Utiliser le provider (ou controller)	Dans cet écran, utiliser le LocationProvider (ou ce que Rôle 3 aura exposé) pour lancer la recherche et afficher résultats / loading / erreur.
Ne pas appeler l'API directement	Comme pour Firebase : tu passes par le state management (provider / controller), pas par <code>location_api.dart</code> directement.
Intégrer dans « Ajouter un service »	Optionnel : bouton « Choisir le lieu » qui ouvre l'écran de recherche et récupère le résultat avec <code>Navigator.pop(context, selectedLocation)</code> .

Une fois que Rôle 4 a livré son API et que Rôle 3 a branché le provider (et les routes), tu n'as plus qu'à brancher ton écran sur ce provider pour que la page affiche correctement le résultat de la personne 4 et que le state management relie tout.

Document complémentaire à : [GUIDE_EQUIPE_AIDLY.md](#)

Pour : Rôle 2 (UI/Design – pages)

En lien avec : Rôle 3 (State management), Rôle 4 (API / Localisation)